

1 Data Pre-Processing

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
[2]: dataset = pd.read_csv("Data.csv")
dataset
```

```
[2]:   Country  Age  Salary Purchased
0  France  44.0  72000.0         No
1   Spain  27.0  48000.0         Yes
2  Germany 30.0  54000.0         No
3   Spain  38.0  61000.0         No
4  Germany 40.0     NaN         Yes
5  France  35.0  58000.0         Yes
6   Spain  NaN  52000.0         No
7  France  48.0  79000.0         Yes
8  Germany 50.0  83000.0         No
9  France  37.0  67000.0         Yes
```

```
[3]: # numpy
X_raw = dataset.iloc[:, :-1].values
y_raw = dataset.iloc[:, -1].values
```

```
[4]: print(X_raw)
print(y_raw)
```

```
[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 nan]
 ['France' 35.0 58000.0]
 ['Spain' nan 52000.0]
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]
<StringArray>
['No', 'Yes', 'No', 'No', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes']
Length: 10, dtype: str
```

```
[5]: # pandas
dfX_raw = dataset.iloc[:, :-1]
seriesy_rawdfX = dataset.iloc[:, -1]
```

```
[6]: print(dfX_raw)
print(seriesy_rawdfX)
```

```
   Country  Age  Salary
0  France  44.0  72000.0
1   Spain  27.0  48000.0
2  Germany 30.0  54000.0
3   Spain  38.0  61000.0
```

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date: 17th August 2026

```

4 Germany 40.0 NaN
5 France 35.0 58000.0
6 Spain NaN 52000.0
7 France 48.0 79000.0
8 Germany 50.0 83000.0
9 France 37.0 67000.0
0 No
1 Yes
2 No
3 No
4 Yes
5 Yes
6 No
7 Yes
8 No
9 Yes

```

Name: Purchased, dtype: str

```

[7]: # copy the data to another variable
dfX = dfX_raw.copy()
seriesy = seriesy_rawdfX.copy()
X = X_raw.copy()
y = y_raw.copy()

```

```

[8]: print(X)

```

```

[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 nan]
 ['France' 35.0 58000.0]
 ['Spain' nan 52000.0]
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]

```

```

[9]: print(type(dfX))
print(type(X))

```

```

<class 'pandas.DataFrame'>
<class 'numpy.ndarray'>

```

```

[10]: print(y)

```

```

<StringArray>
['No', 'Yes', 'No', 'No', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes']
Length: 10, dtype: str

```

```

[11]: print(type(seriesy))
print(type(y))

```

```

<class 'pandas.Series'>
<class 'pandas.arrays.StringArray'>

```

Name: Swapnaraj Mohanty
 SIC No: 24BCSH93
 Lab Roll No: 11
 Date: 17th August 2026

1.1 Taking care of missing data

```
[12]: print(dfX.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0   Country  10 non-null      str
 1   Age      9 non-null       float64
 2   Salary   9 non-null       float64
dtypes: float64(2), str(1)
memory usage: 372.0 bytes
None
```

```
[13]: print(X.info())
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[13], line 1
----> 1 print(X.info())

AttributeError: 'numpy.ndarray' object has no attribute 'info'
```

```
[14]: # change numpy array to dataframe
X = pd.DataFrame(X)
X.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0    0      10 non-null      str
 1    1      9 non-null       object
 2    2      9 non-null       object
dtypes: object(2), str(1)
memory usage: 372.0+ bytes
```

```
[15]: # Then change to numpy
X = X.values
X
```

```
[15]: array([[ 'France', 44.0, 72000.0],
        [ 'Spain', 27.0, 48000.0],
        [ 'Germany', 30.0, 54000.0],
        [ 'Spain', 38.0, 61000.0],
        [ 'Germany', 40.0, nan],
        [ 'France', 35.0, 58000.0],
        [ 'Spain', nan, 52000.0],
        [ 'France', 48.0, 79000.0],
        [ 'Germany', 50.0, 83000.0],
```

```
['France', 37.0, 67000.0]], dtype=object)
```

```
[16]: print(dfX.isnull().sum())
```

```
Country    0
Age         1
Salary      1
dtype: int64
```

```
[17]: print(X.isnull().sum())
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[17], line 1
----> 1 print(X.isnull().sum())

AttributeError: 'numpy.ndarray' object has no attribute 'isnull'
```

```
[18]: dfX_raw
```

```
[18]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
[19]: dfX
```

```
[19]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
[20]: dfX = dfX.fillna(0)
dfX
```

```
[20]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date: 17th August 2026

2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	0.0
5	France	35.0	58000.0
6	Spain	0.0	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
[21]: # replacing 0 with NaN value
```

```
dfX = dfX.replace(0, np.nan)
dfX
```

```
[21]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
[22]: index_no = dfX.columns.get_loc("Age")
index_no
```

```
[22]: 1
```

```
[23]: dfX.iloc[:, 1] = dfX.iloc[:, 1].fillna(5)
dfX
```

```
[23]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	5.0	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
[24]: # getting the old data
dfX = dfX_raw.copy()
dfX
```

```
[24]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0

2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

1.2 Missing value imputation in dataframe

1.2.1 Replace with mean values

```
[25]: dfX_raw
```

```
[25]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
[26]: # dataframe
dfX = dfX_raw.copy()

from sklearn.impute import SimpleImputer    # SimpleImputer is a class ie usedd to
    ↪ handle the misssing values with simple strategies
imputer = SimpleImputer(missing_values=np.nan, strategy="mean")

# imputer.fit(dfX.iloc[:, 1:3])
# dfX.iloc[:, 1:3] = imputer.transform(dfX.iloc[:, 1:3])
# dfX.iloc[:, 1:3] = imputer.fit(dfX.iloc[:, 1:3]).transform(dfX.iloc[:, 1:3])

dfX.iloc[:, 1:3] = imputer.fit_transform(dfX.iloc[:, 1:3])
dfX
```

```
[26]:
```

	Country	Age	Salary
0	France	44.000000	72000.000000
1	Spain	27.000000	48000.000000
2	Germany	30.000000	54000.000000
3	Spain	38.000000	61000.000000
4	Germany	40.000000	63777.777778
5	France	35.000000	58000.000000
6	Spain	38.777778	52000.000000
7	France	48.000000	79000.000000
8	Germany	50.000000	83000.000000
9	France	37.000000	67000.000000

```
[27]: # dataframe
dfX = dfX_raw.copy()

from sklearn.impute import SimpleImputer    # SimpleImputer is a class ie usedd to
      ↪handle the misssing values with simple strategies
imputer = SimpleImputer(missing_values=np.nan, strategy="median")

# imputer.fit(dfX.iloc[:, 1:3])
# dfX.iloc[:, 1:3] = imputer.transform(dfX.iloc[:, 1:3])
# dfX.iloc[:, 1:3] = imputer.fit(dfX.iloc[:, 1:3]).transform(dfX.iloc[:, 1:3])

dfX.iloc[:, 1:3] = imputer.fit_transform(dfX.iloc[:, 1:3])
dfX
```

```
[27]:   Country  Age  Salary
0  France  44.0  72000.0
1   Spain  27.0  48000.0
2  Germany 30.0  54000.0
3   Spain  38.0  61000.0
4  Germany 40.0  61000.0
5  France  35.0  58000.0
6   Spain  38.0  52000.0
7  France  48.0  79000.0
8  Germany 50.0  83000.0
9  France  37.0  67000.0
```

```
[28]: # getting the old data
dfX = dfX_raw.copy()
dfX
```

```
[28]:   Country  Age  Salary
0  France  44.0  72000.0
1   Spain  27.0  48000.0
2  Germany 30.0  54000.0
3   Spain  38.0  61000.0
4  Germany 40.0     NaN
5  France  35.0  58000.0
6   Spain   NaN  52000.0
7  France  48.0  79000.0
8  Germany 50.0  83000.0
9  France  37.0  67000.0
```

```
[29]: # dataframe

from sklearn.impute import SimpleImputer    # SimpleImputer is a class ie usedd to
      ↪handle the misssing values with simple strategies
imputer = SimpleImputer(missing_values=np.nan, strategy="constant", fill_value=5)
imputer.fit(dfX.iloc[:, 1:3])
dfX.iloc[:, 1:3] = imputer.fit(dfX.iloc[:, 1:3]).transform(dfX.iloc[:, 1:3])
dfX
```

```
[29]:   Country  Age  Salary
0  France  44.0  72000.0
```

```

1   Spain  27.0  48000.0
2   Germany 30.0  54000.0
3   Spain  38.0  61000.0
4   Germany 40.0      5.0
5   France 35.0  58000.0
6   Spain   5.0  52000.0
7   France 48.0  79000.0
8   Germany 50.0  83000.0
9   France 37.0  67000.0

```

1.3 Missing value imputaion with numpy

1.3.1 Replace with mean values

```
[30]: X_raw
```

```
[30]: array([[ 'France', 44.0, 72000.0],
        [ 'Spain', 27.0, 48000.0],
        [ 'Germany', 30.0, 54000.0],
        [ 'Spain', 38.0, 61000.0],
        [ 'Germany', 40.0, nan],
        [ 'France', 35.0, 58000.0],
        [ 'Spain', nan, 52000.0],
        [ 'France', 48.0, 79000.0],
        [ 'Germany', 50.0, 83000.0],
        [ 'France', 37.0, 67000.0]], dtype=object)
```

```
[31]: X = X_raw.copy()

from sklearn.impute import SimpleImputer
imputer = SimpleImputer(missing_values=np.nan, strategy="mean")
imputer.fit(X[:, 1:3])
X[:, 1:3] = imputer.fit_transform(X[:, 1:3])
```

```
[32]: print(X)
```

```

[ 'France' 44.0 72000.0]
[ 'Spain' 27.0 48000.0]
[ 'Germany' 30.0 54000.0]
[ 'Spain' 38.0 61000.0]
[ 'Germany' 40.0 63777.77777777778]
[ 'France' 35.0 58000.0]
[ 'Spain' 38.77777777777778 52000.0]
[ 'France' 48.0 79000.0]
[ 'Germany' 50.0 83000.0]
[ 'France' 37.0 67000.0]

```

1.4 Encoding the categorical data

```
[33]: dfX
```

```
[33]: Country  Age  Salary
0  France  44.0  72000.0
1   Spain  27.0  48000.0
2  Germany 30.0  54000.0
3   Spain  38.0  61000.0
4  Germany 40.0     5.0
5  France  35.0  58000.0
6   Spain   5.0  52000.0
7  France  48.0  79000.0
8  Germany 50.0  83000.0
9  France  37.0  67000.0
```

```
[34]: # pandas
dfX = pd.get_dummies(dfX, columns=["Country"]) # one-hot encoding (for each category)
dfX
```

```
[34]:   Age  Salary  Country_France  Country_Germany  Country_Spain
0  44.0  72000.0             True             False             False
1  27.0  48000.0             False             False             True
2  30.0  54000.0             False             True             False
3  38.0  61000.0             False             False             True
4  40.0     5.0             False             True             False
5  35.0  58000.0             True             False             False
6   5.0  52000.0             False             False             True
7  48.0  79000.0             True             False             False
8  50.0  83000.0             False             True             False
9  37.0  67000.0             True             False             False
```

```
[35]: from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
# ct = ColumnTransformer(transformers=[("encode", "drop", [0])],
    ↳remainder="passthrough")
ct = ColumnTransformer(transformers=[("encode", OneHotEncoder(), [0])],
    ↳remainder="passthrough")
# ct = ColumnTransformer(transformers=[("encode", OneHotEncoder(), [0])])
X = np.array(ct.fit_transform(X))
```

```
[36]: print(X)
```

```
[[1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [0.0 1.0 0.0 30.0 54000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 35.0 58000.0]
 [0.0 0.0 1.0 38.77777777777778 52000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

1.5 Encoding the Dependent Variable

```
[37]: from sklearn.preprocessing import LabelEncoder # used to convert the categorical
      ↪ target labels into numerical labels
      le = LabelEncoder()
      y = le.fit_transform(y)
```

```
[38]: print(y)

[0 1 0 0 1 1 0 1 0 1]
```

```
[39]: le.fit(["india", "paris", "paris", "tokyo", "amsterdam"])
      print(le.transform(["india", "amsterdam", "tokyo", "paris"]))

[1 0 3 2]
```

1.6 Splitting the dataset into Training set and Test set

```
[40]: from sklearn.model_selection import train_test_split
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
      ↪ random_state=1) # random-state is used for reproducibility
```

```
[41]: print(X_train)

[[0.0 0.0 1.0 38.77777777777778 52000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 35.0 58000.0]]
```

```
[42]: print(X_test)

[[0.0 1.0 0.0 30.0 54000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

```
[43]: print(y_train)

[0 1 0 0 1 1 0 1]
```

```
[44]: print(y_test)

[0 1]
```

1.7 Features Scaling

```
[45]: from sklearn.preprocessing import StandardScaler # Z_score = [(x - x_bar) / s] or
      ↪ [(x - mu) / sigma]
      sc = StandardScaler() # x -> data
      X_train[:, 3:] = sc.fit_transform(X_train[:, 3:]) # x_bar / mu -> sample /
      ↪ population mean
```

```
X_test[:, 3:] = sc.transform(X_test[:, 3:])          # s / sigma -> sample / population
↳ standard deviation
```

```
[46]: print(X_train)
```

```
[[0.0 0.0 1.0 -0.19159184384578545 -1.0781259408412425]
 [0.0 1.0 0.0 -0.014117293757057777 -0.07013167641635372]
 [1.0 0.0 0.0 0.566708506533324 0.633562432710455]
 [0.0 0.0 1.0 -0.30453019390224867 -0.30786617274297867]
 [0.0 0.0 1.0 -1.9018011447007988 -1.420463615551582]
 [1.0 0.0 0.0 1.1475343068237058 1.232653363453549]
 [0.0 1.0 0.0 1.4379472069688968 1.5749910381638885]
 [1.0 0.0 0.0 -0.7401495441200351 -0.5646194287757332]]
```

```
[47]: print(X_test)
```

```
[[0.0 1.0 0.0 -1.4661817944830124 -0.9069571034860727]
 [1.0 0.0 0.0 -0.44973664397484414 0.2056403393225306]]
```